

# Lab 05 — Turn the Brochures into a SQLite Knowledge Base

---

Agentic AI Applications with Codex · Day 1 · Codex

**Course:** Agentic AI Applications with Codex (C695)

**Day 1 · Topic 2 · about 25 minutes · slides 59–62**

**Surface:** Codex

**Features:** SQLite FTS5 · official SQLite WASM · nested AGENTS.md

## The story so far

A day after going live Grace's phone keeps ringing: is the macaron class nut-free, where do I park, can my 13-year-old come? The answers are all in her brochures. Nobody reads them. Step one of a course assistant: make those documents searchable.

## Your goal

The chatbot must answer from Cook & Bake's own documents. Put them in one searchable SQLite file the browser can load.

## You'll build

data/academy.db — 140 searchable chunks and a courses table

## What is in this folder

- assets/kb/
- assets/brochures-pdf/
- assets/kb-AGENTS.md
- solution/ — reference files from the verified build
- prompts.md / prompts.pdf — every prompt, ready to paste
- evidence/checklist.md — what to capture as proof

## Step by step

1. **Add the documents** — Copy the kb/ folder (20 brochures, FAQ, policies, campuses) into your repo, with kb/AGENTS.md.
2. **Install SQLite WASM** — Run `npm init -y`, then `npm install -D @sqlite.org/sqlite-wasm`.
3. **Ask for the build script** — Paste the prompt.
4. **Build it** — Run `npm run build:kb`. Note the chunk count.
5. **Query it** — Ask Codex to run three test queries and show the top result for each.

## The prompts

### PROMPT — Codex

Write scripts/build-kb.mjs and an npm script "build:kb" that builds data/academy.db.

- Use @sqlite.org/sqlite-wasm (the official build, which includes FTS5) — the same engine the browser will use.
- Split every Markdown file in kb/ into one chunk per "## " section. Keep the document title.
- FTS5 table chunks(doc\_id UNINDEXED, title, section, body, url UNINDEXED) with tokenize='porter unicode61'.
- A normal table courses(...) from data/courses.json for fee and date questions.
- Write the file with sqlite3\_js\_db\_export.

Then test: "refunds", "nut allergy macaron", "Bukit Timah parking" — show the top hit for each.

## Check your work

- ☐ data/academy.db exists (about 150 KB).
- ☐ The build reports about 140 chunks from 23 documents and 20 courses.
- ☐ "refunds" finds the refund policy — porter stemming works.
- ☐ "nut allergy macaron" returns the BAK-104 allergen section.
- ☐ Running the build twice gives the same counts.

## If it goes wrong

- **no such module: fts5** — You installed sql.js, whose default build has no FTS5. Use @sqlite.org/sqlite-wasm.

## Stretch

- Ingest the PDF brochures directly with pdfjs-dist and compare the chunk count with the Markdown build.

**Why it matters:** Why SQLite and not a vector database? One file, no server, runs on GitHub Pages, and bm25 keyword ranking is strong on short, factual course documents.

## Next

Lab 6 — Build the Course Assistant and Drive It with /goal. Keep what you built — the next lab starts from it.

## Safety

Use only this lab's synthetic data and your own accounts. Never paste a real API key, password or personal data into a prompt or a file. Every email address in the data must resolve to your own inbox. Nothing is published or sent without your explicit approval.